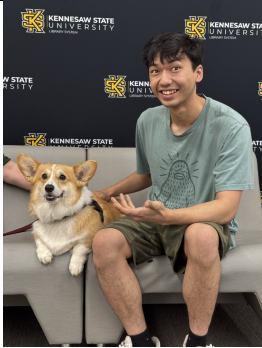
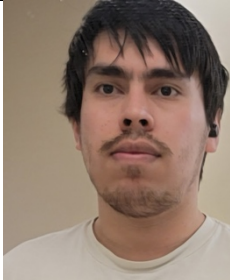


Project Plan
SP-14 - Green - Chess AI
CS 4850 - Section 03 – Fall 2025
Aug 31, 2025

			
Matt Staats Developer	Jason Nguyen Documentation	Carlos Pena Documentation	Chiagoziem Iteogu Developer

Project Team

Roles	Name	Team Function	Contact (Cell Phone) and Alt Email
Project owner or Advisor	Sharon Perry	Facilitate project progress; advise on project planning and management.	770.329.3895 Sperry46@kennesaw.edu
Team leader	Matt Staats	Team Leader & Developer	404.405.6710 matthewcstaats@gmail.com
Team members	Jason Nguyen	Documentation & Testing	404.747.3352 Neilgroup101@gmail.com
	Carlos Pena	Documentation & Website	404.519.5636 Cromero2511@gmail.com
	Chiagoziem Iteogu	Developer	478.955.1662 Goziem.iteogu@yahoo.com

Project Overview / Abstract (Research):

The Objective task is to build a web-based application of visualize game board for chess. As well as creating a game mode to either player vs player, or ai on the game of chess. The engine is run with methodology of neural engine with emphasis on self-play by deep learning and reinforcement learning via synthetic data of self-generated playing against itself (Alpha-Zero style).

Project Website:

Example of our future website:

<https://sites.google.com/site/chess-ai-green>

Deliverables – Specific to Your Project

1. Planning phase:

Project Scope:

- **Create a chess application that offers the following features:**
 - Player vs Player game mode
 - Player vs AI game mode
 - Look into creating an AI vs AI game mode to test different difficulty and game rules against itself.
- **Resources**
 - Use the resources provided to us in the project instructions like Pygame module to properly utilize Python's libraries.
 - Use the minimax algorithm as the base for the decision making of AI with other tools to improve AI's decision-making abilities.

2. Requirement specs:

- Gather and define project functional and non-functional requirements for Software Requirements Specification (SRS) document.
- **Non-Functional Requirements (some examples)**
 - **Maintainability:** The code will use Object-Oriented Programming, modular, and be commented appropriately to keep proper organization.
 - **Compatibility:** The application should be able to run on any operating system with no issues.

- **Reliability:** The game does not make illegal moves and enforces the rules of the game of chess. Also, the game should not crash from fatal errors.
- **Usability:** Users should be able to move pieces with ease if it's a legal move. Invalid moves should reset the piece back to its original position, and prompt or indicate to user that the move was invalid.
- **Functional Requirements (some examples)**
 - Game Modes: Player vs Player and Player vs AI
 - Game Rules: The rules of chess must be enforced and follow the world standard rules for the game. Captured pieces must be removed from the board.
 - UI: Allow users to drag and drop pieces when moving a piece. Provide a chess board that saves the current state and updates the board after each valid move.
 - Reset function: Function used to reset the game after a user has finished their match or has made a mistake.

3. Design Document:

- **Class diagram**
 - Chess Board
 - Chess Pieces (Pawn, Rooks, knights, Bishops, king and queens)
 - Move validator: Checks for legal moves on board
 - Game director: Turns, rules, and win or lose conditions
 - Ai Engine (Minimax algorithm)
 - UI mockup Board and drop pieces

4. Development Document:

- **Project Process**
 - **Project Overview SDLC**
 - Objective and scope of the project
 - Software Development Lifecycle model chosen
 - **Project Plan**
 - Timeline (milestone)
 - Role & Responsibilities
 - Platform & Framework selected
 - **Requirement & Design Documentation**
 - **Requirement**
 - **Functional Requirements**
 - **Non-Functional Requirements**

- **Design**
 - **Flowchart**
 - **Class diagram**
 - **System architecture diagram**
 - **UI design prototypes**
 - **Development Documentation**
 - Model selection
 - Integration Testing Documentation
 - Testing Documentation
 - Debugging info
- **Source codebase documentation**
 - Using GitHub organization account for version control
 - Add proper code documentation into the source code, for readability and maintainability
- **Final Report/ Research paper**
 - Report on the Final Project Document
 - Project Website
 - Presentation Slide
 - Presentation Demo Video

5. Software Testing Plan:

- **Unit Testing**
 - Test each module individually, their functions, methods, and classes.
 - Verify that the functionality is working as intended by itself
- **Full Integration Testing**
 - Test all modules after they have been merged and how they work with each other.
 - Make sure all parts work together and cause no issues or errors.
- **System Testing**
 - Perform functional tests
 - Performance tests to make sure the system is dependable.
 - Stress tests the program to test the performance of the game under not ideal conditions.

6. Prototype

- **Chess AI Training**

- Development of a training pipeline using chess datasets (PGN) and reinforcement/self-play to produce a working model capable of generating legal chess moves.
- **Chess AI Testing and Evaluation Deployment**
 - Automated testing suite for model performance, including self-play, matches against Stockfish, and metric reporting (Elo estimate, win/draw/loss). Deployment of the evaluation workflow to ensure reproducibility and easy iteration.
- **Player UI**
 - A simple user web interface, where players can interact with the trained model, play games, and export PGN records.

7. Final Report Package:

- Website
- Source Code
- Final Report
- Final Presentation Video
- Group Account link to GitHub

8. Weekly Activity Reports (WAR)

- Each member will describe their weekly activities to our advisor. There will be ten reports in total that each member will need to do.

Collaboration and Communication Plan:

Communication for this project will be done through Discord and group text messaging. We will meet weekly on Fridays on our group's server to discuss progress, milestones, tasks, and future milestones. We will also collaborate with each other after each class period on Tuesdays and Thursdays.

Collaboration - Discord for team discussions between status meetings. One Drive to share documents, diagrams, and designs for the documentation aspect. GitHub will be used to work on the codebase together during the project's development life cycle.

Group Meeting Schedule Date/Time:

Future Status Meetings with Advisor:

- RAD Documentation Review on 9/19/2025 Friday 2 PM.
- Development Documentation Review on 10/17/2025 Friday 2 PM.
- Final Draft Review on 11/21/2025 Friday 2 PM.

Prototype Presentation & Peer Reviews:

- 11/4/2025 Tuesday at 2 PM the group will have prototype walkthroughs.
- 11/4/2025 Tuesday at 2 PM the group will have peer review submissions.

Group Meeting Schedule:

Since the group decided on Friday's at 2 PM for our status reviews with our advisor, we decided to keep it simple and make this the official day and time our group meets to discuss weekly activities and progress on current tasks. Our group will mainly use Discord for our weekly group meeting discussions.

Project Schedule and Task Planning:

See the Gantt chart file attached.

Other Plans:

- Possibility of Adding Dynamic AI Adaptability for Different Difficulty Levels
 - Implement and test different AI difficulty levels like Beginner, Medium, Intermediate, and Grand Master (Very Hard).
 - Research ways to create adjustable AI difficulty to match the player's skill level.

Our project's code will be hosted on GitHub, and we will use Git repository for user collaboration and version control. This will allow us to track code changes and to revert to previous versions. Allowing us to collaborate more efficiently with code reviews on pull requests before merging the code into production.

[illegible]